Columnar OLAP Storage Engine

Advanced Database Management — Computer Science (A & B), 4th Semester

Project 02 Manual / *Languages: C or C++*

1 Problem Statement

You will build a small analytical database engine that stores tabular data in a columnar format and answers aggregation queries against it. A columnar store organizes data by column rather than by row: every column of a table lives in its own file, with values stored contiguously, so that a query that reads only two columns out of seven touches only those two files and ignores the rest. This is the design used by Parquet, ClickHouse, DuckDB, Snowflake, BigQuery, and every modern data warehouse. It is almost the opposite of the row-oriented storage you used in Project 01 with PostgreSQL and understanding why both designs exist, and when each wins, is the main point of this project.

The reason columnar storage is faster for analytical queries comes in two parts. First, I/O reduction: a query like `SELECT AVG(price) FROM sales WHERE year = 2024` on a seven-column table needs only the `price` and `year` columns. A row store has to read every byte of every row anyway, because the rows are interleaved on disk. A column store reads only two files and ignores the other five, cutting I/O by 70%. Second, compression: values within a single column are usually similar to each other, a `country` column has maybe 24 distinct values repeated millions of times, and a `date` column sorted by day has long runs of the same value. Per-column compression can shrink these by 50% or more. Row-oriented storage cannot exploit either of these properties because it interleaves unrelated types of data on every page.

The tradeoff is that point lookups (`SELECT * WHERE id = 42`) become slower in a column store, because reconstructing a single row means reading seven separate files. This is one of the teaching gotchas of the project: after learning at how fast `SUM(price) GROUP BY country` runs, you will also observe that a single-row lookup is slower than the row-oriented baseline. The conclusion is that row storage and column storage are not rivals but complements, each optimized for a different workload. A real modern company usually runs both.

You will build the full pipeline: a CSV loader that ingests row-oriented data and writes it out as per-column binary files, two column encoding schemes (dictionary and run-length encoding) applied automatically based on the data, a tiny query executor that knows how to scan only the needed columns and apply predicates and aggregates, and a benchmark that compares your column store against a naive row-oriented baseline on the same machine with the same data. What you will *not* build is a SQL parser, joins, updates, deletes, indexes, or transactions. The scope is load-once, query-many — which is exactly how real analytical workloads work.

2 What the Final Product Looks Like

When your project is complete, you will be able to load a CSV file, watch the engine decide which encoding to use for each column, and run aggregation queries that read only the columns they need.

```
$ make
$ ./colsh --data ./warehouse

colsh> LOAD sales.csv AS sales
Loaded 1,000,000 rows, 7 columns into ./warehouse/sales/

  column name     type     encoding     size
  -----------     ----     --------     -----------
  id              int64    none         8.0 MB
  date            int32    rle          0.3 MB  (365 runs)
  country         string   dict         0.2 MB  (24 distinct)
  category        string   dict         0.3 MB  (87 distinct)
  product_id      int32    none         4.0 MB
  quantity        int32    none         4.0 MB
  price           double   none         8.0 MB
```

```
  total on disk:             24.8 MB
  original CSV size:         68.2 MB
  compression ratio:         2.75x

colsh> QUERY SELECT country, SUM(price) FROM sales
              WHERE date >= 20240101 GROUP BY country
 country          | sum_price
 -----------------+-------------
 United States    |  4521923.45
 United Kingdom   |  1245123.10
 Germany          |   987432.20
 ...
 (24 rows, 38ms)
 [read 3 of 7 columns, 8.5 MB from disk]

colsh> QUERY SELECT COUNT(*) FROM sales WHERE category = 'Electronics'
 count
 -------
 82134
 (1 row, 12ms)
 [read 1 of 7 columns, 0.3 MB from disk]

colsh> QUERY SELECT * FROM sales WHERE id = 500000
 id      | date     | country         | category     | product_id | quantity |
     price
 --------+----------+-----------------+--------------+------------+----------+-------

 500000  | 20240614 | Germany         | Books        | 7712       | 2        |
     34.50
 (1 row, 47ms)
 [read 7 of 7 columns, 24.6 MB from disk]

 note: point lookups read every column, which is slower than a row store.
 compare against: ./rowbench lookup sales id=500000

colsh> BENCH run
Running benchmark against both engines on the same data...

Query 1: SELECT SUM(price) WHERE date >= 20240101
  column store: 28ms  (read 2 columns,  8.3 MB)
  row    store: 412ms (read every row, 68.2 MB)
  speedup: 14.7x

Query 2: SELECT country, AVG(quantity) GROUP BY country
  column store: 35ms  (read 2 columns,  4.2 MB)
  row    store: 389ms
  speedup: 11.1x

Query 3: SELECT * WHERE id = 500000
  column store: 47ms  (read 7 columns, 24.6 MB)
  row    store: 28ms  (read 1 row seek,   88  B)
  speedup: 0.60x  <-- row store wins

colsh> \quit
```

The three queries in this session tell the whole story. The first two are analytical: they scan millions of rows but touch only a few columns, and the column store is roughly ten times faster because it physically reads less data. The third is a point lookup: the column store has to read seven files to reconstruct one row, while the row store does a single seek. The column store *loses* this query, by design. This is the defining tradeoff of columnar storage, and making it visible to students is the pedagogical heart of the project.

3 Architectural Overview

The engine has four components. Keep them strictly separated.

```
     [ CSV file ]                      [ query from shell ]
          |                                   |
          v                                   v
    +-----------+                     +----------------+
    |  Loader   |                     |  Query parser  |
    |  (Sec 5)  |                     |  (tiny DSL)    |
    +-----+-----+                     +-------+--------+
          |                                   |
          v                                   v
    +-----------------------+         +------------------+
    |  Column writers       |         |  Query executor  |
    |  - encoding chooser   |         |  - column reader |
    |  - dict builder       |         |  - predicate     |
    |  - rle encoder        |         |  - aggregator    |
    |  - binary serializer  |         |  - group by      |
    +-----------+-----------+         +---------+--------+
                |                               ^
                v                               |
         +--------------+                +-------------+
         |  column      |---------------->|  column     |
         |  files on    |   reads only    |  readers    |
         |  disk        |   needed cols   |  (Sec 6)    |
         +--------------+                +-------------+
```

The loader is the write path: it ingests a CSV, decides per-column encoding, and writes binary files. The query executor is the read path: it parses a query, opens only the files it needs, scans them, and produces results. The two paths share a common file format (Section 4) but otherwise do not interact.

4 The Column File Format

Each column of each table lives in its own binary file inside a per-table directory. The layout of a single column file is:

```
Offset  Size       Field            Description
------  ---------  ---------------  --------------------------
  0     4          magic            ASCII bytes 'C','O','L','1'
  4     4          version          uint32, must be 1
  8     1          data_type        uint8 (see below)
  9     1          encoding         uint8 (0=none, 1=dict, 2=rle)
 10     2          reserved         uint16, must be 0
 12     8          row_count (N)    uint64, total rows in column
 20     8          data_size        uint64, size of data section
 28     8          dict_size        uint64, size of dict (0 if n/a)
 36     ...        data section     N values, encoded
 ...    ...        dict section     dictionary (only if encoding=1)
 ...    8          footer_crc64     CRC64 over everything above
```

Data types:

- `1 = INT32`: 4-byte signed little-endian integers.
- `2 = INT64`: 8-byte signed little-endian integers.
- `3 = DOUBLE`: 8-byte IEEE 754 binary64.
- `4 = STRING`: variable-length, stored as a 2-byte length prefix followed by bytes (only used in the dictionary and in uncompressed string columns).

The on-disk layout of the `data section` depends on the encoding chosen for the column, explained in Section 5.

5 Loading and Encoding

The loader reads a CSV file, infers each column's type from the header or from a schema file, decides an encoding per column, and writes the resulting binary files. Directory structure for a loaded table:

```
./warehouse/sales/
    schema.json      <- list of columns with name, type, encoding
    id.col
    date.col
    country.col
    ...
```

The `schema.json` file is a simple text file (JSON or a plain key-value format you define) that lists every column in table order, with its name, type, and encoding. The query executor reads this file first to know what columns exist and which files to open.

5.1 Encoding Selection

For each column, the loader chooses one of three encodings. The rules are deterministic and simple:

Dictionary encoding. If the column is a `STRING` type, use dictionary encoding. The loader scans the entire column, builds a map from distinct string to small integer id (0 through $D - 1$ where D is the number of distinct values), and writes the data section as N integers (the ids, not the strings) plus a dictionary at the end mapping id back to string. If the column has more than 65,535 distinct values, fall back to no encoding (store the raw strings); for this project such columns are rare and the fallback is acceptable.

The integer ids are stored as either 1 byte (if $D \leq 256$), 2 bytes (if $D \leq 65{,}536$), or 4 bytes. Pick the smallest that fits. Document the chosen width in the column file header by extending the reserved byte — or just always use 2 bytes if you prefer simplicity.

Run-length encoding (RLE). If the column is numeric and the loader detects that the values form long runs of identical values (for example, a sorted `date` column has 365 distinct values but millions of rows, so each value repeats thousands of times), use RLE. The data section becomes a sequence of `(value, run_length)` pairs. A run whose length is 1 is still encoded as a pair, which costs extra space compared to raw storage, so the loader should only choose RLE if the number of runs is less than about $N/4$ (otherwise the encoding costs more than it saves). The decision is made by doing a quick first pass over the column and counting runs.

No encoding. If neither of the above fires, store the values as raw binary one after another. This is the fallback for high-cardinality numeric columns (e.g., `price`) where compression would not help.

5.2 The Write Path

The end-to-end load sequence for a CSV file:

1. Read the CSV header. Determine each column's name and type. (The type can come from a schema file or be inferred: if every value parses as an integer, it is an integer; if not but every value parses as a double, it is a double; otherwise it is a string.)
2. Read all rows into row-oriented memory. Yes, this uses a lot of RAM temporarily; for the datasets in this project (millions of rows, hundreds of MB) a modern laptop has no trouble. Streaming approaches are a possible bonus.
3. For each column, run the encoding-selection logic above to decide an encoding.
4. For each column, construct the data section in memory: either raw values, dictionary-encoded ids, or RLE pairs.
5. Write each column to its own file using the format in Section 4. Use the temp-file-and-rename pattern so a crash mid-load does not leave corrupt column files.
6. Write the `schema.json` file last, also using temp-and-rename.

The schema file is written last on purpose: if it exists, all the column files it references are guaranteed to exist. A reader that fails to open `schema.json` treats the table as not present; a reader that successfully opens it can trust every referenced column file.

6 The Query Executor

The query executor is the read path. It accepts a query in the tiny DSL described in Section 7, figures out which columns it needs, opens only those column files, scans them while applying the predicate,

and aggregates the results.

6.1 The Column Reader

For each encoding, you need a reader function that returns an iterator of decoded values:

- **No encoding**: read the data section as-is, one value at a time. The value type is known from the column header.
- **Dictionary**: read the dictionary into memory (it is small). Then read the data section as small integers; for each integer, look up the corresponding string in the dictionary.
- **RLE**: read the data section as `(value, length)` pairs. The iterator emits `value` `length` times before advancing to the next pair.

The iterator abstraction is what makes everything downstream uniform. The filter and the aggregator do not care how the column was encoded; they just pull values one at a time.

6.2 Predicate Evaluation

A query may have a `WHERE` clause with a simple predicate: `column OP value`, where `OP` is one of `=`, `<`, `<=`, `>`, `>=`, `!=`. Only one predicate is supported (no `AND`, `OR`); multiple predicates are a bonus.

The executor reads the predicate's column and walks it, producing a bitmap of which row positions satisfy the predicate: a bit vector of length N where bit i is 1 if row i passed the filter. The bitmap is then used to select matching values when scanning the other columns.

6.3 Aggregation

The supported aggregate functions are `COUNT(*)`, `COUNT(col)`, `SUM(col)`, `AVG(col)`, `MIN(col)`, `MAX(col)`. For `GROUP BY column`, the executor reads the grouping column, builds a hash map from group key to a running aggregate state, and walks every row that passed the filter, updating the appropriate group's state.

For `SELECT *` (no aggregate), the executor reads every column, applies the filter, and prints the matching rows reconstructed from the per-column values. This is the "point lookup" path that the column store is deliberately bad at.

7 The Query DSL

To avoid writing a SQL parser (which is a project of its own), the engine accepts a small subset of SQL with a fixed grammar:

```
QUERY   := SELECT select_list FROM table_name
           [WHERE predicate] [GROUP BY column]

select_list := STAR
             | expr (, expr)*
expr    := column | agg_function(column) | agg_function(*)

agg_function := COUNT | SUM | AVG | MIN | MAX

predicate := column op literal
op        := = | < | <= | > | >= | !=
literal   := integer | double | 'string'
```

Hand-write a small recursive descent parser for this grammar. It is about 100 lines of C. The parser produces a small in-memory query representation (a struct listing the selected expressions, the source table, an optional predicate, and an optional group-by column) which the executor then runs.

8 Phase 1 — CSV Loader and No-Encoding Columns

The goal of Phase 1 is an end-to-end pipeline from CSV to queryable columns, but with only the "no encoding" format supported. No dictionary, no RLE. This gets the loader, the file format, the column reader, and the query executor all working before adding the complications of encoding.

Build these components:

- A CSV parser that handles basic quoting and commas.
- Type inference over the rows.
- The column file writer for raw (uncompressed) columns of each type.
- The column file reader for the same.
- The schema file writer and reader.
- The query DSL parser for a limited subset: `SELECT` of columns or `COUNT(*)` or `SUM(col)`, `FROM`, and a single `WHERE` predicate.
- The executor loop: open the columns you need, apply the filter, compute the aggregate or print the rows.
- The `colsh` shell with `LOAD` and `QUERY` commands.

At the end of Phase 1, loading a CSV and running `SELECT SUM(price) WHERE year = 2024` should work, and should demonstrably read only two columns from disk. You can verify this by instrumenting the column reader to log which files it opens.

9 Phase 2 — Dictionary and RLE Encodings

The goal of Phase 2 is to add both encodings and the automatic encoding selector.

Build these components:

- The dictionary builder: scan a string column, find distinct values, assign ids, produce an in-memory mapping.
- The dictionary-encoded writer: emit the ids as small integers in the data section, then the dictionary itself at the end.
- The dictionary-encoded reader: load the dictionary on file open, then decode ids to strings as they stream.
- The run-length encoder: detect runs in a numeric column, emit `(value, length)` pairs.
- The RLE decoder.
- The encoding-selection logic in the loader.
- `GROUP BY` support in the executor, since dictionary columns are the common input to `GROUP BY`.

At the end of Phase 2 the full example session in Section 2 should work except for the benchmark. Verify with the per-column size breakdown that dictionary-encoded string columns are dramatically smaller than they would be raw, and that RLE columns on sorted data are nearly free.

10 Phase 3 — Row-Oriented Baseline and Benchmark

The goal of Phase 3 is to build a minimal row-oriented storage engine alongside the column store and produce the benchmark from Section 2 showing where each design wins.

The row-oriented baseline is deliberately simple: a single binary file per table, with rows stored one after another, each row being a length-prefixed concatenation of its column values. There is no indexing, no block structure, no compression. Loading a CSV into this format is straightforward. Queries scan the file from start to finish, decoding each row, applying the filter, and aggregating. This is roughly 30–50 lines of code per query type; do not overengineer it.

Build these components:

- A row writer that reads a CSV and emits the flat row file.
- A row reader that scans the file and applies a predicate.
- The same query DSL, executed by scanning rows instead of columns.
- A `BENCH run` command in the shell (or a separate binary) that loads a fixed benchmark dataset into both engines and runs the three queries from the example session, measuring and printing wall-clock times for each.

10.1 The Benchmark Dataset

The benchmark uses a synthetic `sales` table with 1,000,000 rows and the seven columns shown in the example session. Generate it programmatically (a small C or C++ program that writes the CSV to stdout using `rand()` with a fixed seed). The seed is fixed so that every group's benchmark runs against

identical data. The exact data generation logic is up to you; a reasonable approach is: `id` sequential, `date` random over a year, `country` picked uniformly from 24 options, `category` picked from 87 options, `product_id` uniform in 1..10000, `quantity` uniform in 1..10, `price` a reasonable double.

10.2 Required Benchmark Queries

Run these three queries on both engines and report the times:

1. `SELECT SUM(price) FROM sales WHERE date >= 20240101` — a two-column analytical query. Expected: column store at least 5x faster than row store.
2. `SELECT country, AVG(quantity) FROM sales GROUP BY country` — a two-column grouped aggregate. Expected: column store at least 5x faster.
3. `SELECT * FROM sales WHERE id = 500000` — a single-row point lookup. Expected: row store is faster, and your design document must discuss why.

11 Deliverables and Submission

11.1 What to Submit

A single zip archive containing:

- The complete source tree of both engines (column store, row-oriented baseline), the shell, and the benchmark driver.
- A `Makefile` (or `CMakeLists.txt`) that builds everything with one command on a recent Ubuntu LTS.
- A `README.md` with build instructions, supported commands, and limitations.
- A `design.pdf` of at most 10 pages describing: the column file format, the encoding-selection logic, the query executor's column-iterator model, the predicate and group-by implementation, the row-oriented baseline, and a detailed discussion of the benchmark results including an explanation of why query 3 favors the row store.
- A `benchmark/` directory with the benchmark driver, the CSV generator, and the output of your final run.
- A `tests/` directory with tests for the CSV parser, each encoding reader/writer round trip, the predicate evaluator, the group-by aggregator, and an end-to-end test that loads a small table and verifies the query results are correct.

11.2 Archive Naming

`Group[Number]_Project02_Columnar.zip`

11.3 Demonstration

Each group will present a live demonstration. First, a walkthrough of the column file format and the encoding selection using the design document. Second, a live session with the shell: load the benchmark dataset, run the three required queries, run the benchmark, and show the size breakdown by column. The instructor is free to propose additional queries on the spot. Third, a short oral examination in which each group member will be asked to explain, without notes, a specific component, with particular focus on why columnar storage loses the point-lookup query and why it wins the aggregation queries. All group members must be present.

12 Bonus Opportunities

Each independent. Bonus credit is awarded on top of the base grade up to a maximum of 15 percentage points.

12.1 Bit-Packing and Delta Encoding (up to 5%)

Add two more encodings. Bit-packing stores integer columns in the minimum number of bits needed (if every value fits in 12 bits, use 12 bits per value instead of 32). Delta encoding stores a monotonically increasing sequence as the first value followed by the successive differences, which are usually much smaller than the absolute values and compress well when combined with bit-packing. Both are used heavily in Parquet. Measure the compression improvement and the decoding speed.

12.2 Multi-Predicate Queries (up to 5%)

Extend the DSL and executor to support multiple predicates combined with `AND` and `OR`. The natural implementation is to produce a bitmap for each predicate and combine them with bitwise operations before scanning the aggregate columns. Add at least three new benchmark queries that exercise multi-predicate filtering.

12.3 Streaming CSV Loader (up to 5%)

The base project reads the entire CSV into memory before writing the column files. Implement a streaming loader that processes the CSV in chunks (e.g., 100,000 rows at a time), writing out partial column files per chunk and merging them at the end. Demonstrate loading a dataset larger than available RAM. Describe the merging strategy in the design document.

13 Constraints and Prohibitions

The Following Will Result in Loss of Credit

- Use of any existing columnar, analytical, or data warehouse library as a component. This includes but is not limited to Apache Parquet (any language binding), Apache Arrow, DuckDB, ClickHouse, Apache ORC, and their bindings. The column file format must be your own.
- Use of any existing CSV parsing library. Write your own; a basic one is under 50 lines.
- Use of any existing SQL parser. The tiny DSL parser must be hand-written.
- Use of any existing compression library (zlib, snappy, zstd, lz4). The two required encodings (dictionary, RLE) must be implemented from scratch; bit-packing and delta in the bonus must also be written by hand.
- Use of any existing binary serialization library (Protobuf, FlatBuffers, Cap'n Proto). Direct reads and writes only.
- Storing columns as rows on disk. The column-per-file layout is required. This is the one thing the project is about.
- Use of Python at any layer, including tests and benchmark drivers. Tests and scripts must be in C, C++, shell, or JavaScript.
- Submitting code whose commit history shows large, infrequent commits. The incremental nature of the engine should be visible.

14 Required Reading

The first item is required reading before you start. The second is required before Phase 2. The remaining items are reference material.

1. Abadi, D., Madden, S., and Hachem, N. "Column-Stores vs. Row-Stores: How Different Are They Really?" *ACM SIGMOD*, 2008. The canonical paper comparing the two architectures on the same workloads. Read Sections 1 through 3 before Phase 1; they describe exactly the tradeoffs your benchmark will demonstrate.
2. Abadi, D., Madden, S., and Ferreira, M. "Integrating Compression and Execution in Column-Oriented Database Systems." *ACM SIGMOD*, 2006. The paper that describes RLE, dictionary, and bit-packing encodings in the context of column stores. Sections 2 and 3 are the relevant ones; read before Phase 2.
3. Kleppmann, M. *Designing Data-Intensive Applications*, Chapter 3, section "Column-Oriented Storage." The clearest modern introduction to the topic, with diagrams.
4. The Apache Parquet documentation, `https://parquet.apache.org/docs/file-format/`. Reference material describing the format used by production data warehouses. You are not implementing Parquet, but understanding its structure is useful context.
5. Stonebraker, M. et al. "C-Store: A Column-oriented DBMS." *VLDB*, 2005. The original paper that launched the modern column-store era. Optional but historically interesting.

Penalty Structure

- Missing documentation: –10%
- Security vulnerabilities: –5% per issue
- Poor code quality: –5% to –15%
- Academic integrity violations: Zero grade

Contact & Support

Teaching Assistants:
Jalal Ahmed: `bscs23134@itu.edu.pk`
Khadijah Farooqi: `bscs23128@itu.edu.pk`

Office Hours: Monday & Thursday,
11:30 AM – 2:30 PM